

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1 Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2 Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3 Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where X : Input Image, K : Kernel (Filter), Y : Feature Map.

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where N = Input Size, F = Filter Size, P = Padding, S = Stride.

3.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map.

1. Identity Kernel Preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose: Preserves the original image; used for understanding convolution.

2. Sobel-X Kernel (Vertical Edge Detection)

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications: Object detection, lane detection, face recognition.

3. Sobel-Y Kernel (Horizontal Edge Detection)

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications: Text detection, boundary extraction, medical image analysis.

4. Laplacian Kernel

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications: Edge enhancement, satellite image analysis, medical imaging.

5. Sharpening Kernel

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications: Image enhancement, OCR, face recognition.

6. Box Blur Kernel

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications: Noise removal, image smoothing, preprocessing.

7. Gaussian Blur Kernel

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications: Noise reduction, computer vision preprocessing, feature extraction.

8. Emboss Kernel

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications: Texture analysis, artistic image effects.

9. Outline Detection Kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications: Image segmentation, boundary detection, shape extraction.

10. Motion Blur Kernel

Simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications: Motion simulation, image restoration, video processing.

3.2 Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges (all directions)	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4 Numerical Example 1: Convolution

Input image:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

First position: $1(1) + 2(0) + 4(0) + 5(1) = 6$

Second position: $2(1) + 3(0) + 5(0) + 6(1) = 8$

Third position: $4(1) + 5(0) + 7(0) + 8(1) = 12$

Fourth position: $5(1) + 6(0) + 8(0) + 9(1) = 14$

Feature Map:

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5 Numerical Example 2: Max Pooling

Feature map:

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter:

$$\begin{array}{ll} \textbf{Window 1:} \begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8 & \textbf{Window 2:} \begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3 \\ \textbf{Window 3:} \begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6 & \textbf{Window 4:} \begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9 \end{array}$$

Output:

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6 Numerical Example 3: Parameter Calculation

Input Image: $32 \times 32 \times 3$

Convolution Layer: Filters = 16, Kernel = 3×3

$$\text{Parameters} = (3 \times 3 \times 3 + 1) \times 16 = (27 + 1) \times 16 = 448$$

Therefore, total trainable parameters = 448.

7 Dataset

Dataset: CIFAR-10

- Training Images: 50,000
- Testing Images: 10,000
- Classes: 10
- Image Size: $32 \times 32 \times 3$

8 Experimental Procedure

Task 1 Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2 Implement a convolution layer. Compare Kernel = 3×3 . Record feature map sizes.

Task 3 Study hyperparameters. Compare Stride = 1, Stride = 2, Padding = Same, Padding = Valid. Compute output dimensions.

Task 4 Visualize feature maps after the first convolution layer. Display at least eight feature maps.

Task 5 Compare Max Pooling and Average Pooling. Compare output size and accuracy.

Task 6 Construct the following CNN:

Input $\rightarrow$ Conv2D₃₂(3×3 , ReLU) $\rightarrow$ MaxPool(2×2) $\rightarrow$ Conv2D₆₄(3×3 , ReLU) $\rightarrow$ MaxPool(2×2) $\rightarrow$ Flatten $\rightarrow$ Dense₁₀

Train for: Optimizer – Adam, Epochs – 20, Batch Size – 32,64.

Task 7 Evaluate Accuracy, Precision, Recall, F1-score, Confusion Matrix, Classification Report.

9 Source Code

A Convolutional Neural Network (CNN) was built and trained to classify images into multiple classes by using TensorFlow/Keras in a Jupyter Notebook. The project covers loading images, pre-processing images, creating a CNN model and training it, evaluating the trained model based on accuracy, precision, recall, F1-score, and confusion matrix, and visualizing the training and validation metrics. Moreover, convolutional feature maps have been extracted and visualized in this project. Besides, the comparison between Max Pooling and Average Pooling is included.

The full source code can be found at:

https://github.com/Mohula-Prasath/Deep_learning_lab/tree/main/lab-3

10 Experimental Results

10.1 Dataset Description

The experiment for multi-class classification was performed using CIFAR-10 database. This database contains colorful images classified into ten different categories. There are 50,000 images in the training database and 10,000 images in the testing database. Size of each image is 32×32 pixels with three RGB color channels.

The dataset used for the experiment has the following characteristics:

- Number of Training Samples: 50,000
- Number of Testing Samples: 10,000
- Image Size: 32×32 pixels
- Number of Color Channels: 3 (RGB)
- Number of Classes: 10

10.2 Data Preprocessing

The images in CIFAR-10 were normalized in terms of pixel intensity values before the CNN model was trained. The original range of pixel intensity values was from 0 to 255. To perform Min-Max normalization, the pixel values were divided by 255. Thus, the range of pixel values became 0 to 1.

The class labels were retained in integer form since the `sparse_categorical_crossentropy` loss function was used during model training.

The final dataset used for training had the following characteristics:

- Feature Scaling: Pixel normalization to the range $[0, 1]$
- Training Samples: 50,000
- Testing Samples: 10,000
- Validation Split: 20% of the training data
- Input Shape: $32 \times 32 \times 3$
- Output Classes: 10

10.3 CNN Model Training

Convolutional Neural Network (CNN) was created as the baseline architecture for classification of multi-class images. The architecture consists of two convolutional layers, followed by a max pooling layer after each of them. The first convolutional layer has 32 filters, while the second one has 64 filters. The 3×3 kernel size and ReLU activation were chosen for both convolutional layers.

Flattened feature maps were then fed into a fully connected layer consisting of 128 neurons with ReLU activation. The 0.5 dropout was used to minimize overfitting. Finally, the output dense layer with 10 neurons and Softmax activation was utilized for classifying the images into ten CIFAR-10 classes.

The model was optimized with the Adam optimizer at a learning rate of 0.001 using the sparse categorical cross-entropy as the loss function. The training process was performed for 20 epochs, batch size 64, and a validation split of 20%.

Baseline CNN architecture is presented below:

- Convolutional Layers: 2
- Convolutional Filters: 32 and 64
- Kernel Size: 3×3
- Activation Function: ReLU
- Pooling Method: Max Pooling
- Fully Connected Neurons: 128
- Dropout Rate: 0.5
- Output Activation: Softmax
- Number of Output Classes: 10
- Optimizer: Adam
- Learning Rate: 0.001
- Batch Size: 64
- Number of Epochs: 20
- Validation Split: 20%

10.4 Feature Map Visualization

In order to study the internal representations obtained in the CNN model, feature maps obtained from the convolutional layers were analyzed for a selected test image. These feature maps were shown in order to understand the features obtained by different convolutional filters.

The first convolutional layer outputs 32 feature maps, and the second convolutional layer outputs 64 feature maps. This shows how the CNN model extracts features in the input image through convolutional layers.

10.5 Max Pooling and Average Pooling Comparison

A further CNN architecture was trained after replacing Max Pooling layers of the initial model with Average Pooling layers, while keeping all other aspects of the model architecture the same. This was done to compare the performance of both pooling layers. Both models were trained for 20 epochs using Adam optimizer with a learning rate of 0.001, a batch size of 64, and a validation split of 20%.

Comparison of two models was done based on their test accuracy, test loss, precision, recall, and F1-score.

Configuration of both models is listed below:

- Baseline Pooling: Max Pooling
- Comparison Pooling: Average Pooling
- Optimizer: Adam
- Learning Rate: 0.001
- Batch Size: 64
- Number of Epochs: 20
- Dropout Rate: 0.5

10.6 Model Evaluation

Evaluations on the trained model were carried out using unseen test datasets. Accuracy, precision, recall, and F1 score were some of the metrics used for the evaluation of the models. Confusion matrices and classification reports were also produced.

The final evaluation results are presented below.

- **Max Pooling Model**
 - Accuracy: 0.7101
 - Precision: 0.7182130743602934
 - Recall: 0.7101
 - F1-Score: 0.7118567024381257
 - Test Loss: 0.9059980511665344
- **Average Pooling Model**
 - Accuracy: 0.7102
 - Precision: 0.7091765010337954
 - Recall: 0.7101999999999999
 - F1-Score: 0.7079680814601003
 - Test Loss: 0.8631484508514404

The confusion matrices and classification reports were used to identify the classes that were classified correctly and the classes that resulted in misclassification. Training and validation accuracy and loss curves were also analyzed to understand the learning behavior of both models and to identify potential signs of overfitting.

The comparison between Max Pooling and Average Pooling provides insight into how the choice of pooling operation affects the feature extraction process and the overall classification performance.

11 Mandatory Plots

1. Sample Images

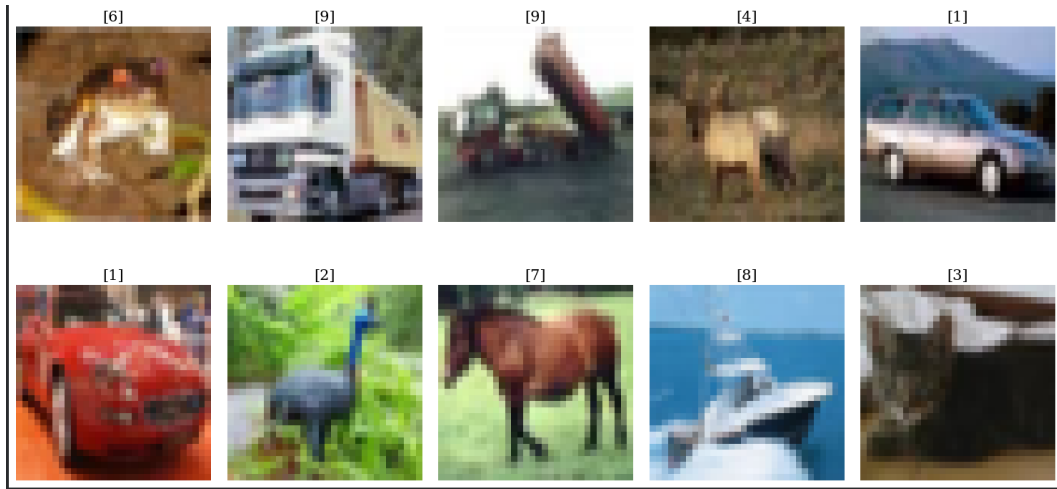


Figure 1: Sample Images from the CIFAR-10 Dataset

Inference: The above picture provides some of the images that belong to the CIFAR-10 dataset. They are images of objects belonging to various classes and are used in multi-class classification.

2. Class Distribution

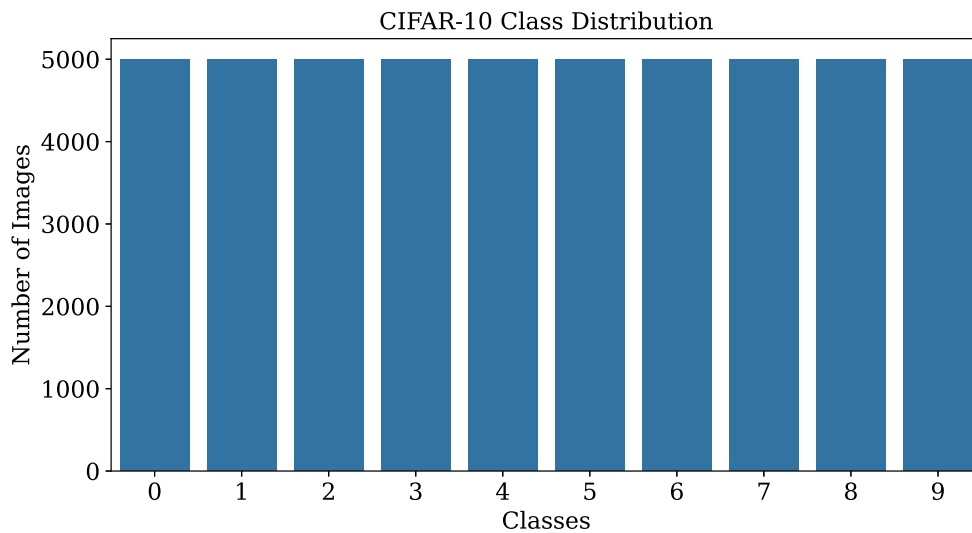


Figure 2: Class Distribution of the CIFAR-10 Dataset

Inference: Class distribution indicates that CIFAR-10 database has equal numbers of images belonging to all ten classes. Hence, there is no class imbalance within the data set.

3. Training and Validation Accuracy

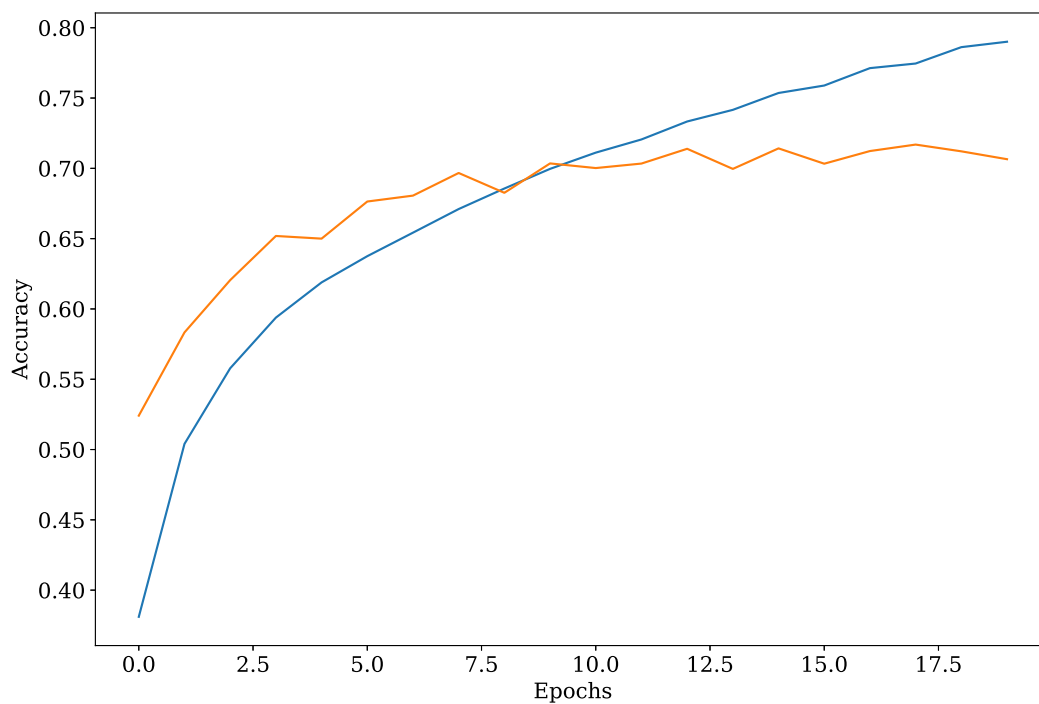


Figure 3: Training and Validation Accuracy

Inference: Accuracy on the training set rises steadily as the CNN learns from the training data. Accuracy on the validation set determines the ability of the network to generalize to unseen validation data. the validation accuracy rises but starts flattening out at the end.

4. Training and Validation Loss

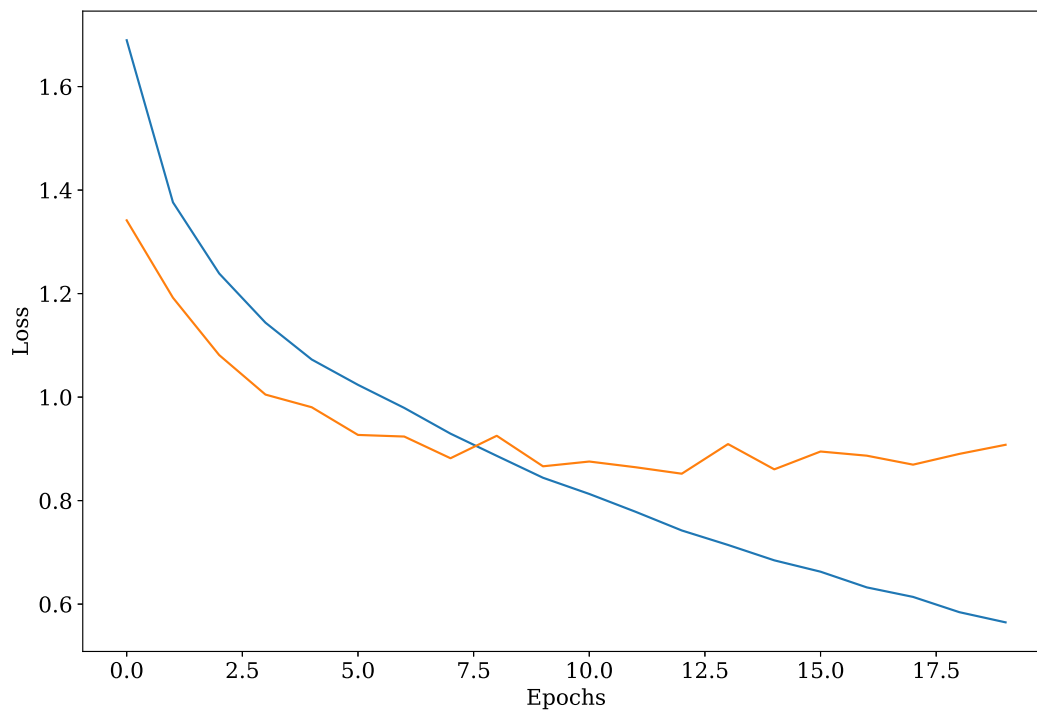


Figure 4: Training and Validation Loss

Inference: The training loss tends to reduce as the model is trained to avoid misclassifications. Validation loss reduces easily at first but later flattens out.

5. Feature Maps

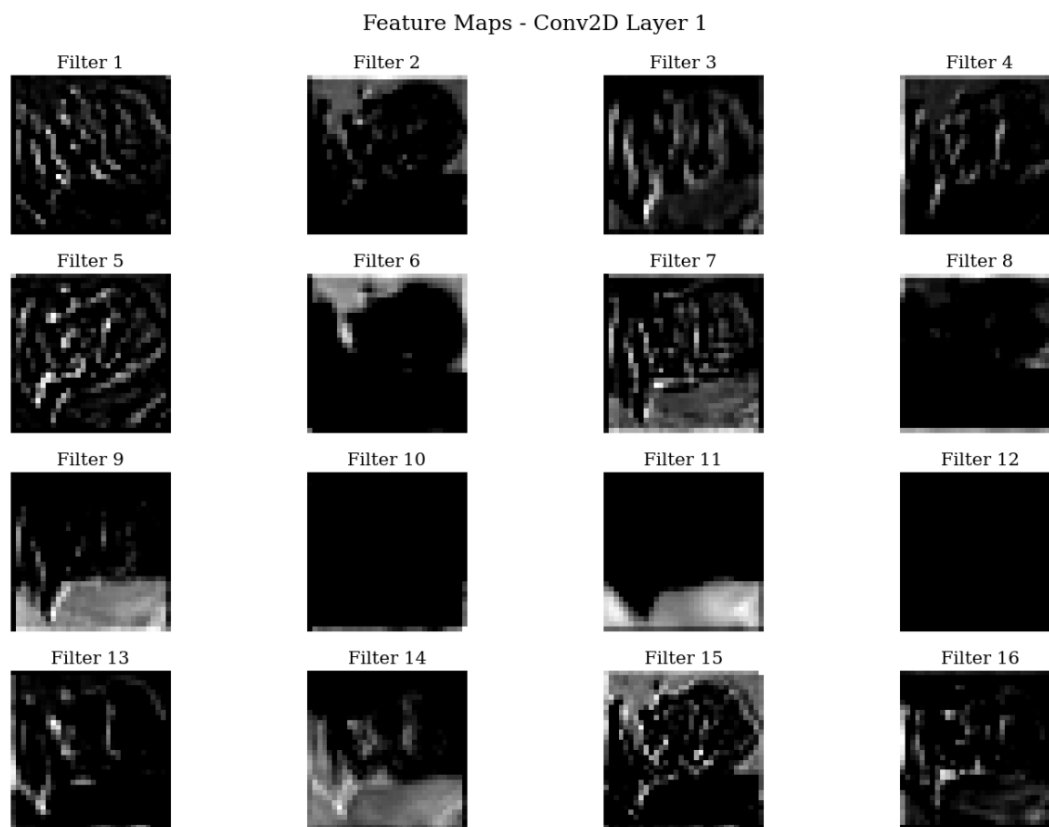


Figure 5: Feature Maps from the First Convolutional Layer

Inference: The feature maps indicate the different features that have been identified through the filters used in the first convolutional layer. The filters are used to identify low-level features from the input image.

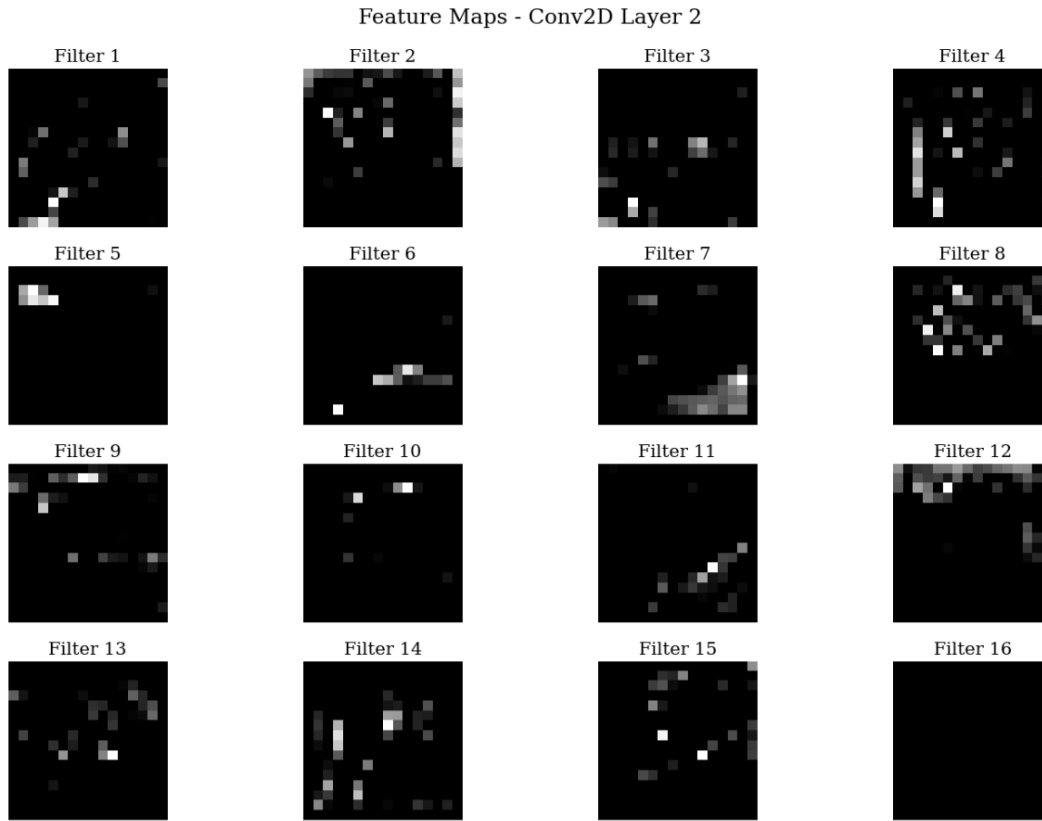


Figure 6: Feature Maps from the Second Convolutional Layer

Inference: The second convolutional layer creates high-level feature representations depending on the features created in the previous layer. The feature maps show how the CNN learns complex features from the image input.

6. Confusion Matrix

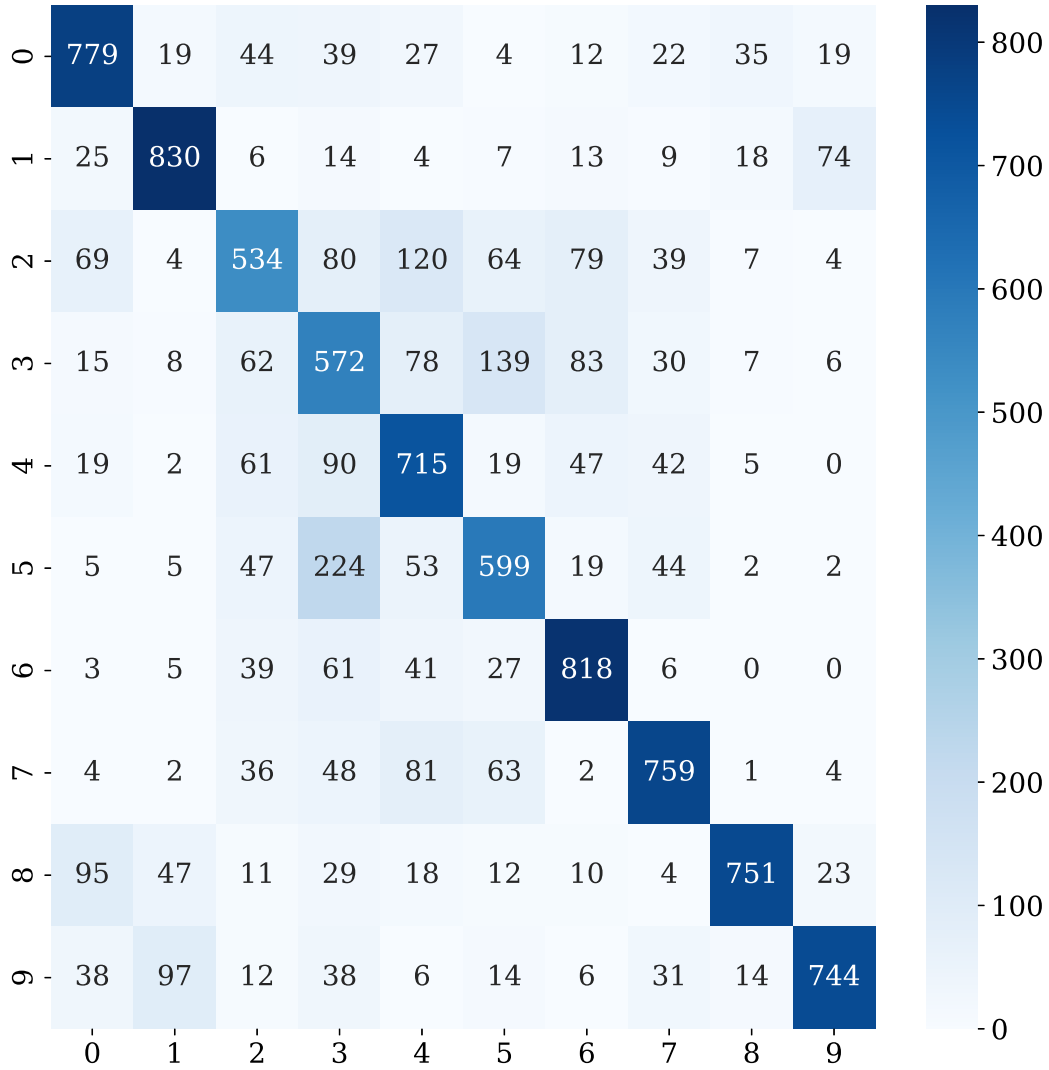


Figure 7: Confusion Matrix of the CNN Model

Inference: The confusion matrix reveals the results obtained by the CNN in terms of classifying each of the ten classes of CIFAR-10. The higher the values on the diagonal, the greater the number of images that have been correctly classified. From this we could clearly see the model confuses a lot between cat and dog

12 Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

The output size of a convolution layer is calculated using:

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1$$

where:

- $N = 64$ is the input size,
- $K = 5$ is the kernel size,
- $S = 2$ is the stride,
- $P = 2$ is the padding.

Substituting the values:

$$O = \left\lfloor \frac{64 + 2(2) - 5}{2} \right\rfloor + 1$$

$$O = \left\lfloor \frac{63}{2} \right\rfloor + 1$$

$$O = 31 + 1 = 32$$

Therefore, the output spatial dimensions are:

$$\boxed{32 \times 32}$$

Hence, the convolution operation produces a 32×32 feature map for each filter.

2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.

An RGB image has 3 input channels. Each filter therefore has dimensions:

$$3 \times 3 \times 3$$

The number of weights for one filter is:

$$3 \times 3 \times 3 = 27$$

Since there are 64 filters, the total number of weights is:

$$27 \times 64 = 1728$$

Each filter also has one bias term. Therefore, the total number of bias parameters is:

$$64$$

Hence, the total number of trainable parameters is:

$$1728 + 64 = 1792$$

Therefore:

$$\boxed{1792 \text{ trainable parameters}}$$

3. Compare ReLU and Sigmoid activation functions.

The Rectified Linear Unit (ReLU) and Sigmoid are two commonly used activation functions in neural networks.

The ReLU function is defined as:

$$f(x) = \max(0, x)$$

while the Sigmoid function is defined as:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Their characteristics can be compared as follows:

Property	ReLU	Sigmoid
Output Range	$[0, \infty)$	$(0, 1)$
Formula	$\max(0, x)$	$\frac{1}{1+e^{-x}}$
Computation	Simple	More computationally expensive
Vanishing Gradient	Less likely for $x > 0$	More likely
Common Usage	Hidden layers	Binary output layers

Table 1: Comparison between ReLU and Sigmoid activation functions

The ReLU activation function is mostly used in hidden layers of CNNs since it is easy to compute and helps minimize the vanishing gradient problem for positive inputs. The sigmoid function restricts the values between 0 and 1 and is often helpful in probability-based classifications.

4. Replace Max Pooling with Average Pooling and compare the results.

In the original CNN network, Max Pooling was applied after each convolutional layer. In the second CNN model, Average Pooling was implemented in place of Max Pooling without altering any other part of the network.

Below are the outcomes of the test-set classification for both CNN models.

Metric	Max Pooling	Average Pooling
Accuracy	0.71	0.71
Macro Precision	0.72	0.71
Macro Recall	0.71	0.71
Macro F1-Score	0.71	0.71

Table 2: Comparison of Max Pooling and Average Pooling on the test set

The final training and validation results after 20 epochs were:

Metric	Max Pooling	Average Pooling
Training Accuracy	0.7901	0.7552
Validation Accuracy	0.7065	0.7180
Training Loss	0.5648	0.6715
Validation Loss	0.9079	0.8536

Table 3: Final training and validation results after 20 epochs

The Max Pooling model got a higher final training accuracy of 79.01% when compared to 75.52% for Average Pooling. Nevertheless, Average Pooling outperformed Max Pooling on validation dataset, giving 71.80% accuracy against 70.65% for the former. Also, Average Pooling gave a lower validation loss of 0.8536 when compared to 0.9079 of Max Pooling.

For the test set, both models provided an accuracy score of about 71%. The Max Pooling model provided slightly higher macro precision of 0.72 when compared to Average Pooling (0.71), with both of them having similar macro recall and F1-score (0.71).

Hence, even though Max Pooling fit the training data better, Average Pooling performed slightly better on validation dataset and had a lower validation loss.

13 Results

13.1 Training Summary

Parameter	Value
Dataset Size	50,000 train / 10,000 test, 10 classes, $32 \times 32 \times 3$
Normalization	Pixel values scaled to $[0, 1]$
Optimizer	Adam
Batch Size	64
Epochs	20
Total Trainable Parameters	545,098
Final Training Accuracy	79.01%
Final Validation Accuracy	70.65%
Test Accuracy	71.01%
Test Loss	0.905998
Precision (weighted)	0.7182
Recall (weighted)	0.7101
F1-score (weighted)	0.7119

Table 4: Training Summary of the CNN Model

13.2 Epoch-wise Training Progress

Epoch	Train Accuracy	Train Loss	Val Accuracy	Val Loss
1	0.3810	1.6896	0.5241	1.3416
2	0.5040	1.3763	0.5833	1.1920
3	0.5578	1.2388	0.6205	1.0812
4	0.5938	1.1438	0.6519	1.0049
5	0.6189	1.0726	0.6500	0.9804
6	0.6375	1.0237	0.6764	0.9269
7	0.6543	0.9791	0.6806	0.9238
8	0.6711	0.9294	0.6967	0.8818
9	0.6856	0.8865	0.6826	0.9253
10	0.6996	0.8441	0.7035	0.8663
11	0.7112	0.8129	0.7002	0.8755
12	0.7206	0.7785	0.7034	0.8645
13	0.7333	0.7423	0.7139	0.8520
14	0.7416	0.7143	0.6996	0.9093
15	0.7536	0.6846	0.7142	0.8604
16	0.7589	0.6626	0.7033	0.8949
17	0.7713	0.6323	0.7123	0.8868
18	0.7746	0.6139	0.7169	0.8695
19	0.7862	0.5844	0.7121	0.8903
20	0.7901	0.5648	0.7065	0.9079

Table 5: Epoch-wise Training Progress for Max Pooling CNN

13.3 Max Pooling vs. Average Pooling Comparison

Pooling Type	Test Accuracy	Test Loss
Max Pooling	0.7101	0.905998
Average Pooling	0.7102	0.863148

Table 6: Max Pooling vs. Average Pooling Comparison

Output shape after one pooling layer (identical for both):

(1, 16, 16, 3)

14 Discussion

1. Why is convolution preferred over fully connected layers for images?

Convolutional layers preserve the spatial structure of images and learn local patterns such as edges, textures, and shapes. They use shared filters across the image, which greatly reduces the number of parameters compared to fully connected layers.

2. How does stride affect the feature map size?

Stride determines how many pixels the convolutional filter moves at each step. A larger stride skips more pixels and produces a smaller feature map, while a smaller stride produces a larger feature map.

3. What is the role of padding?

Padding adds extra pixels, usually zeros, around the border of the input image. It helps preserve spatial dimensions and allows edge information to be processed effectively by the convolutional filters.

4. Why is pooling used?

Pooling reduces the spatial dimensions of feature maps, thereby decreasing computational cost and the number of parameters in subsequent layers. It also helps the network retain important features while providing some robustness to small changes in their position.

5. How do feature maps represent image characteristics?

Feature maps show the responses of convolutional filters to different patterns in an image. Early layers generally detect simple features such as edges and textures, while deeper layers combine these features to represent more complex shapes and objects.

6. Why do CNNs require fewer parameters than MLPs?

CNNs use local connectivity and weight sharing, meaning the same filter weights are applied across different locations of an image. In contrast, an MLP connects every input pixel to neurons in the next layer, resulting in a much larger number of parameters.

15 Conclusion

In this experiment, a working knowledge of Convolutional Neural Network (CNN) for multi-class classification of images has been obtained through the application of the CIFAR-10 dataset. The CNN was able to extract relevant features from the input image due to the use of convolution and feature maps. The impact of convolution, activation function, padding, stride, and pooling were investigated through practical work and experimentation. Max Pooling vs Average Pooling was compared and contrasted. The experiment revealed why CNNs are so efficient in image classification with lower parameters than MLPs.

16 References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.